

AGRIGUARDIAN

AI-POWERED SMART PLANT DISEASE DIAGNOSIS AND DECISION SUPPORT SYSTEM

Youvatheesh Sai S - 24619235

Nidhin Krishna R -24619220

Boobesh S -24619207

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

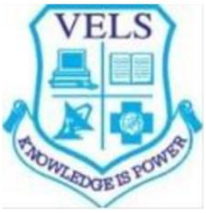
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

2026

BONAFIDE CERTIFICATE

Certified that this project report entitled "AGRIGUARDIAN – AI-POWERED SMART PLANT DISEASE DIAGNOSIS AND DECISION SUPPORT SYSTEM" is the bonafide work of Youvatheesh Sai S [24619235], Nidhin Krishna R [24619220], Boobesh S [24619207] submitted in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Artificial Intelligence and Machine Learning.

The work presented in this report was carried out under the guidance of the project supervisor and within the academic requirements of the Department of Artificial Intelligence and Machine Learning.

SIGNATURE OF THE HOD

Dr.K.Ulaga Priya M.E.,Ph.D (Ph .D)
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR

Ms. F. Benasir Begum M.E,
Project Supervisor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the project work and viva-voice examination held on ……

at Vels Institute of Science, Technology and Advanced Studies (VISTAS),

Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to the management of Vels Institute of Science, Technology and Advanced Studies (VISTAS), the School of Engineering, and the Department of Artificial Intelligence and Machine Learning for providing the academic environment and facilities required to complete this mini project.

We thank our project guide for valuable guidance, technical suggestions, continuous review, and encouragement throughout the development of AgriGuardian. We also acknowledge the support of faculty members and friends who contributed suggestions during the design, implementation, testing, and documentation stages.

Finally, we thank our families for their encouragement and support during the completion of this work.

PROJECT TEAM
Youvatheesh Sai S – 24619235
Nidhin Krishna R – 24619220
Boobesh S – 24619207

TABLE OF CONTENTS

S.No	CONTENTS	PAGE NO.
1	ABSTRACT	i
2	LIST OF FIGURES	ii
3	LIST OF TABLES	iii
4	ABBREVIATIONS	iv
5	**CHAPTER 1: INTRODUCTION**	**1**
5.1	Introduction	2
5.2	Problem Statement	4
5.3	Objectives	5
5.4	Scope	6
5.5	System Overview	7
5.6	Project Contribution	9
6	**CHAPTER 2: LITERATURE REVIEW**	**11**
6.1	Literature Overview	12
6.2	Literature Survey	13
6.3	Research Gap	18
6.4	Summary	20
7	**CHAPTER 3: SYSTEM DESIGN**	**21**
7.1	Existing System	22
7.2	Proposed System	24
7.3	System Architecture	27
7.4	Data Flow	29
7.5	Module Design	31
7.6	Hardware Requirements	36
7.7	Software Requirements	37
8	**CHAPTER 4: IMPLEMENTATION**	**39**
8.1	Methodology	40
8.2	Implementation	45
8.3	Testing and Validation	57
8.4	Integration	60
9	**CHAPTER 5: CONCLUSION AND FUTURE WORK**	**63**
9.1	Conclusion	64
9.2	Limitations	65
9.3	Source Code	67
9.4	Results	76
10	**REFERENCES**	**82**

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
Figure 1	Proposed AgriGuardian system architecture	27
Figure 2	Overall data flow of AgriGuardian	29
Figure 3	Plant identification and disease diagnosis pipeline	31
Figure 4	Severity estimation pipeline	33
Figure 5	Recommendation and decision-support workflow	34
Figure 6	AI What-If Decision Simulator workflow	35
Figure 7	FastAPI backend module structure	36
Figure 8	AgriGuardian user interface – image analysis	76
Figure 9	Disease diagnosis result screen	77
Figure 10	Severity and risk assessment screen	78
Figure 11	Recommendation screen	79
Figure 12	AI assistant / RAG interaction screen	80

LIST OF TABLES

TABLE NO	TITLE	PAGE NO.
Table 1	Problem-to-solution mapping	5
Table 2	Literature comparison	18
Table 3	Research gap	19
Table 4	Hardware requirements	36
Table 5	Software requirements	37
Table 6	Functional modules	38
Table 7	Test cases	58
Table 8	Demonstration results template	81

ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
CV	Computer Vision
OpenCV	Open Source Computer Vision Library
API	Application Programming Interface
RAG	Retrieval-Augmented Generation
HSV	Hue, Saturation, Value
U-Net	U-shaped Convolutional Network
RGB	Red, Green, Blue
HTTP	Hypertext Transfer Protocol
UI	User Interface
DFD	Data Flow Diagram
JSON	JavaScript Object Notation
REST	Representational State Transfer
CPU	Central Processing Unit
GPU	Graphics Processing Unit
RAM	Random Access Memory
VRAM	Video Random Access Memory

ABSTRACT

AgriGuardian is an AI-powered smart plant disease diagnosis and decision support system designed for users who want to grow plants but have limited knowledge of plant health management. The system combines computer vision, machine learning, image preprocessing, disease analysis, severity estimation, risk assessment, recommendations, and a question-answering assistant within a beginner-friendly workflow.

The proposed system accepts plant or leaf images and applies image preprocessing to improve the quality of the visual input. Plant identification and disease diagnosis are performed through an AI-based analysis pipeline. Where lesion-level analysis is available, segmentation techniques such as U-Net or HSV-based processing can isolate affected regions and support severity estimation. The system then converts the diagnosis into practical decision support by presenting treatment choices, prevention measures, and plant-care recommendations.

AgriGuardian further extends diagnosis into an intelligent advisory layer. A grounded agricultural Retrieval-Augmented Generation (RAG) assistant can answer plant-related questions using a controlled knowledge source, while the AI What-If Decision Simulator allows a user to explore possible outcomes of different care decisions. The backend is organized using FastAPI and modular routers for AI analysis, assistant services, recommendations, and plant-related operations.

The project therefore moves beyond simple disease classification by connecting visual diagnosis with severity, risk, recommendation, and interactive decision support. The implementation is intended to provide an accessible foundation for beginners while retaining a modular architecture that can be extended with additional crops, diseases, datasets, models, and mobile interfaces.

CHAPTER 1. INTRODUCTION

1.1 INTRODUCTION

Plants are an important part of food production, home gardening, landscaping, and environmental sustainability. However, maintaining healthy plants requires knowledge about species identification, symptoms, diseases, environmental conditions, nutrient requirements, and appropriate treatment. Beginners frequently find it difficult to distinguish between disease symptoms, nutrient deficiencies, pest damage, and normal variations in plant appearance.

Traditional plant diagnosis generally depends on personal experience, agricultural experts, books, extension services, or online searches. These approaches can be useful but may become slow or confusing when the user does not know the name of the plant or the technical terminology associated with its symptoms. A visual AI system can reduce this initial barrier by allowing a user to submit an image and receive a structured interpretation.

AgriGuardian addresses this requirement through a unified AI-based workflow. Instead of returning only a disease label, the system is designed as a decision support system. It can identify a plant, analyze possible disease conditions, estimate severity where supported by the visual evidence, estimate risk, and provide actionable recommendations. The system also provides preventive guidance and interactive assistance so that users can understand what to do next.

The project is implemented as a modular web-based application with a FastAPI backend. The architecture separates computer vision, assistant, recommendation, and plant-related operations so that individual components can be improved independently. This modular design also supports a future transition from the demonstration website to a dedicated mobile application.

1.2 PROBLEM STATEMENT

Many people are interested in growing plants but do not have sufficient knowledge to identify plants, recognize disease symptoms, estimate the seriousness of an infection, or decide what action should be taken. Existing solutions may focus on classification alone and may not explain the next practical step in a beginner-friendly manner.

The problem addressed by AgriGuardian is therefore the development of an AI-powered smart plant health system that can transform a plant image into an understandable diagnosis and decision-support response while integrating disease assessment, severity analysis, risk estimation, recommendations, prevention, and interactive plant-related assistance.

PROBLEM	AGRIGUARDIAN RESPONSE
Unknown plant or crop	Plant identification module
Unclear visual symptoms	AI-based disease diagnosis
No indication of seriousness	Severity estimation where supported
Uncertainty about future risk	Risk assessment layer
Difficulty choosing treatment	Personalized recommendation engine
Lack of prevention knowledge	Preventive care suggestions

Difficulty asking technical questions	RAG-based plant assistant
Uncertainty about possible actions	AI What-If Decision Simulator

1.3 OBJECTIVES

1. To develop an AI-powered system for plant and leaf image analysis.
2. To identify the plant or crop represented in an input image where the trained model supports the class.
3. To detect probable plant disease conditions using computer vision and machine learning.
4. To estimate disease severity from affected regions when sufficient visual evidence is available.
5. To provide risk-oriented decision support rather than only a disease label.
6. To generate beginner-friendly recommendations, including preventive and treatment-oriented options.
7. To provide an agricultural question-answering assistant using grounded knowledge retrieval.
8. To provide an AI What-If Decision Simulator for exploring alternative plant-care actions.
9. To expose the functionality through a modular FastAPI backend suitable for future mobile integration.

1.4 SCOPE AND OBJECTIVE

The scope of AgriGuardian covers image-based plant health analysis and supporting decision services. The system is primarily intended as an educational and decision-support tool for beginners, hobby growers, and users who need a first-level interpretation before consulting an agricultural expert.

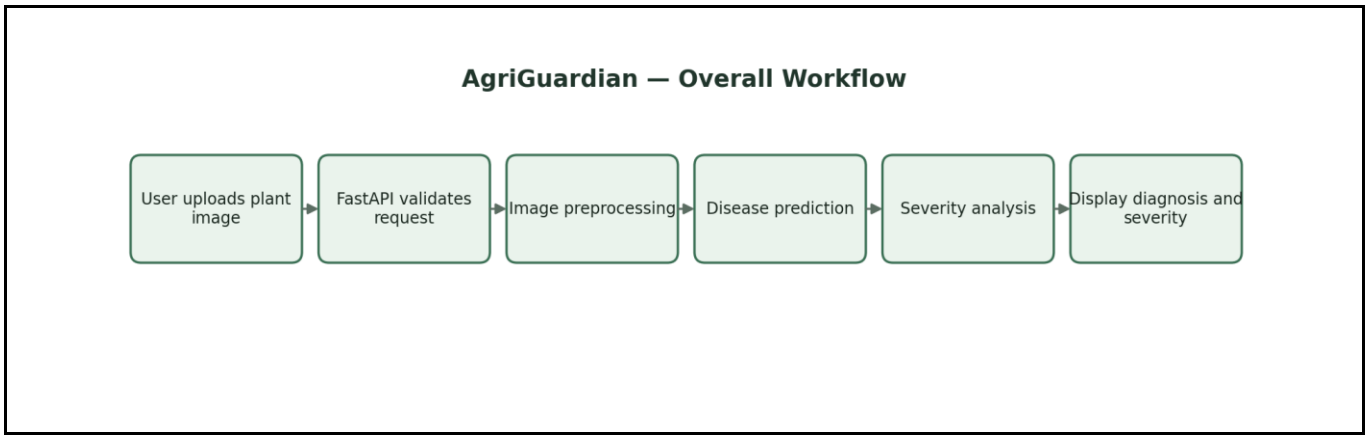
The visual pipeline can be expanded to support additional plant classes and diseases as suitable datasets and validated models become available. The recommendation layer can also be extended by crop, growth stage, environmental conditions, and user-provided observations. The assistant can be improved through a larger verified knowledge base and better retrieval strategies.

The current project should not be interpreted as a replacement for a qualified agricultural diagnosis. Image quality, lighting, occlusion, disease similarity, and dataset limitations can affect predictions. Recommendations should therefore be presented as decision support and not as an absolute guarantee of treatment success.

1.5 SYSTEM OVERVIEW

The system begins with a user-provided plant image. The image is received by the application and passed through preprocessing operations such as resizing,

normalization, and quality preparation. The AI analysis layer then performs plant identification and disease diagnosis according to the available model configuration.
For supported disease conditions, a segmentation or lesion-analysis component can estimate the proportion or distribution of affected regions. This information can be combined with the diagnostic output to derive a severity category. The decision-support layer uses the diagnosis, severity, and contextual information to generate recommendations and prevention guidance.
The RAG assistant operates as a separate service so that general plant-care questions can be answered using a grounded agricultural knowledge source. The What-If simulator uses user-selected actions and scenario parameters to explain possible outcomes and trade-offs. FastAPI provides the service layer connecting these modules with the website interface.



1.6 PROJECT CONTRIBUTION

- Unified workflow connecting plant identification, disease diagnosis, severity, risk, and recommendations.
- Modular AI backend that separates visual analysis from assistant and recommendation services.
- Beginner-oriented presentation of technical disease information.
- Optional lesion segmentation for more informative severity estimation.
- Grounded question-answering architecture for plant-care queries.
- Interactive What-If decision support to move beyond static prediction.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Computer vision has increasingly been applied to agricultural problems such as crop monitoring, disease recognition, weed detection, fruit grading, and plant identification. Convolutional neural networks are particularly useful for extracting visual patterns from images because their learned filters can capture texture, edges, shapes, and higher-level representations.

Plant disease recognition research commonly uses curated image datasets and deep learning classifiers. Transfer learning is frequently considered when the available project dataset is limited because a model pretrained on a large image corpus can provide a useful representation that is later adapted to the agricultural classes.

Image segmentation provides another level of analysis. Instead of assigning one label to an entire image, segmentation attempts to locate pixels or regions associated with lesions or affected tissue. This can support severity estimation when the segmentation output is sufficiently accurate.

Recent intelligent agricultural systems increasingly combine prediction with recommendations. Such systems can incorporate environmental information, crop stage, historical observations, or knowledge bases. Retrieval-Augmented Generation is also relevant to agricultural assistants because it allows natural-language answers to be grounded in a controlled set of documents rather than relying entirely on free-form generation.

2.2 LITERATURE SURVEY

AREA	TECHNIQUE	PURPOSE	OBSERVATION	AGUARDIAN ROLE
Plant disease image classification	Deep learning/CNN	Visual disease classification	Strong pattern recognition; dependent on dataset quality	AgriGuardian uses classification as one stage rather than the complete solution
Transfer learning for crop disease recognition	Pretrained CNN models	Classification with limited data	Useful when project datasets are constrained	Can be incorporated into the diagnosis module
Plant lesion segmentation	U-Net / semantic segmentation	Localization of affected tissue	Enables region-level analysis	Used conceptually for severity estimation
Classical image processing	OpenCV, HSV	Color/texture and region extraction	Fast and interpretable for selected conditions	Used for preprocessing and optional lesion analysis
Agricultural recommendation systems	Rule/knowledge based decision support	Treatment and prevention guidance	Converts prediction into action	AgriGuardian adds personalized beginner-oriented output
RAG-based assistants	Retrieval + language generation	Question answering	Improves grounding and traceability	Applied to plant-related user questions

2.3 RESEARCH GAP

The reviewed approaches show that plant disease classification, image segmentation, image preprocessing, and agricultural advisory systems are often treated as separate components. A classification model may identify a disease without estimating severity; a knowledge system may provide advice without considering visual evidence; and a recommendation system may not expose the uncertainty or risk associated with the prediction.

IDENTIFIED GAP	LIMITATION	AGRI GUARDIAN RESPONSE
Disease label only	Does not directly answer how serious the condition is.	Add severity-oriented visual analysis.
Separate diagnosis and advice	Users must manually search for treatment information.	Connect diagnosis to recommendation services.
Technical outputs	Predictions may be difficult for beginners to interpret.	Use beginner-friendly decision support.
Uncontrolled conversational answers	General AI responses can be inconsistent with agricultural evidence.	Use grounded retrieval for plant questions.
Static recommendations	A single recommendation does not show alternative decisions.	Provide What-If scenario simulation.
Limited modularity	Tightly coupled prototypes are harder to extend.	Use FastAPI routers and separate service modules.

2.4 SUMMARY

The literature indicates that computer vision and deep learning can support plant disease recognition, while segmentation and classical image processing can provide additional spatial information. However, a practical beginner-oriented system benefits from an integrated architecture that links diagnosis to severity, risk, recommendation, and knowledge-based assistance. AgriGuardian is designed around this integration.

CHAPTER 3 – SYSTEM DESIGN

3.1 EXISTING SYSTEM

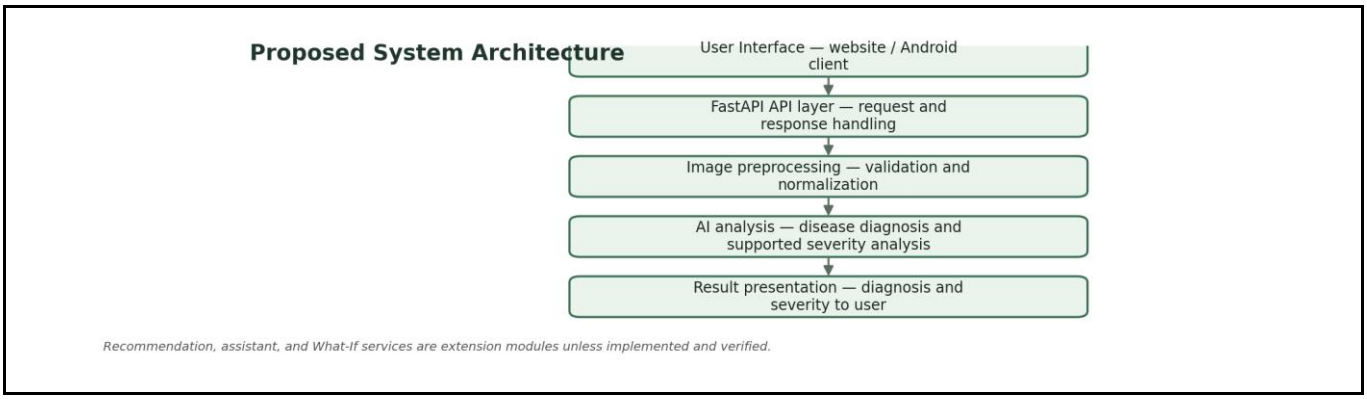
Existing plant-care workflows often require users to identify the plant manually, search for symptoms, compare images, and then determine an appropriate action. Dedicated disease classifiers reduce part of this effort, but a classifier alone does not necessarily provide a complete decision-support workflow.

- Manual identification of plant and disease.
- Dependence on external search or expert consultation.
- Limited severity interpretation.
- Recommendations may be disconnected from the diagnosis.
- No integrated scenario simulation.
- Difficulty for beginners in interpreting technical disease terminology.

3.2 PROPOSED SYSTEM

AgriGuardian proposes a modular pipeline in which visual analysis is connected to decision support. The user supplies an image and receives an analysis containing the supported plant/disease prediction, confidence or diagnostic information where available, severity information where supported, risk-oriented interpretation, and recommendations.

The architecture is designed so that the AI analysis layer, recommendation engine, assistant, and plant services can evolve independently. The FastAPI backend provides API endpoints to the frontend, while the frontend is responsible for user interaction and presentation.



3.3 SYSTEM ARCHITECTURE

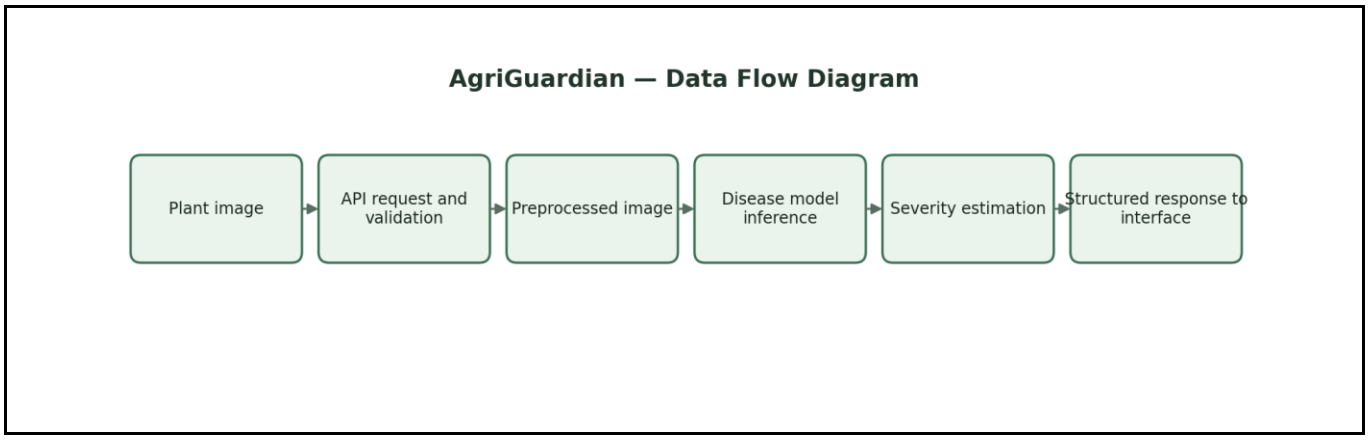
MODULE	FUNCTION

1. User Interface	Accepts image input, displays diagnosis, severity, recommendations and assistant responses.
2. API Layer	FastAPI application manages HTTP requests and connects the frontend to backend services.
3. Image Preprocessing	OpenCV-based preparation, resizing, normalization and quality checks.
4. Plant Identification	Identifies the supported plant/crop class.
5. Disease Diagnosis	Runs the trained AI model and produces a disease-related prediction.
6. Lesion / Severity Analysis	Uses segmentation or HSV-based region analysis where supported.
7. Risk Assessment	Maps diagnostic and severity information into an interpretable risk category.
8. Recommendation Engine	Generates treatment, care and prevention suggestions.
9. RAG Assistant	Retrieves relevant agricultural knowledge and generates grounded answers.
10. What-If Simulator	Compares possible care decisions and explains likely outcomes.

3.4 DATA FLOW

The main data flow starts when a user submits an image. The frontend sends the image to the FastAPI service. The backend validates the request and forwards the image to preprocessing. The processed image is then passed to the appropriate AI analysis component.

The resulting prediction is combined with any available severity or lesion information. The decision-support layer converts the technical output into a response suitable for the user. Recommendations and prevention guidance are then returned to the frontend. Assistant queries follow a separate retrieval path and are not required for the image diagnosis pipeline.



3.5 MODULE DESIGN

MODULE	DESCRIPTION
AI Vision Module	Image preprocessing, plant identification and disease diagnosis.
Severity Module	Affected-region analysis and severity categorization where supported.
Risk Module	Interprets diagnostic/severity signals into risk-oriented guidance.

Recommendation Module	Treatment options, care guidance and preventive measures.
Assistant Module	Plant-related question answering using grounded retrieval.
What-If Module	Scenario-based comparison of alternative plant-care decisions.
Plant Module	Plant information, identification metadata and plant-care details.
API Integration Module	FastAPI routes and request/response handling.
Frontend Module	User interaction, image upload and result visualization.

3.6 HARDWARE REQUIREMENTS

COMPONENT	MINIMUM / DEVELOPMENT REQUIREMENT
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	8 GB minimum; 16 GB recommended
Storage	At least 20 GB available for development assets; SSD recommended
GPU	Optional for inference; dedicated GPU is useful for local model training
Camera	Optional device camera for future image capture
Display	Standard monitor/laptop display
Internet	Required for selected external knowledge/services; local inference can reduce dependence

3.7 SOFTWARE REQUIREMENTS

SOFTWARE	DESCRIPTION
Operating System	Windows 10/11 or Ubuntu-based development environment
Programming Language	Python 3.x
Backend Framework	FastAPI
Computer Vision	OpenCV
Machine Learning	Framework corresponding to the trained AI model
Segmentation	U-Net or HSV-based processing where implemented
Frontend	HTML/CSS/JavaScript or the project frontend used in the demonstration
API Format	HTTP/JSON
Development IDE	Visual Studio / Visual Studio Code or equivalent
Version Control	Git recommended for team development

CHAPTER 4 - IMPLEMENTATION

4.1 METHODOLOGY

The implementation follows a modular pipeline rather than treating the project as a single prediction script. Each major function is represented as a service or router so that the project can be extended without rewriting the complete application.

1. Requirement analysis and definition of beginner-oriented plant health functions.
2. Image acquisition and validation.
3. OpenCV-based preprocessing.
1. Plant identification and disease diagnosis.
1. Optional lesion segmentation and severity analysis.
1. Risk estimation and decision-support generation.
1. Recommendation generation.
1. Grounded RAG assistant.
1. What-If decision simulation.
1. FastAPI integration and frontend communication.
1. Functional testing and demonstration.

4.1.1 IMAGE PREPROCESSING

Image preprocessing is used to convert a user-provided image into a form suitable for the AI pipeline. Typical operations include reading the image, checking its dimensions, resizing it to the model input size, normalizing pixel values, and handling channels consistently.

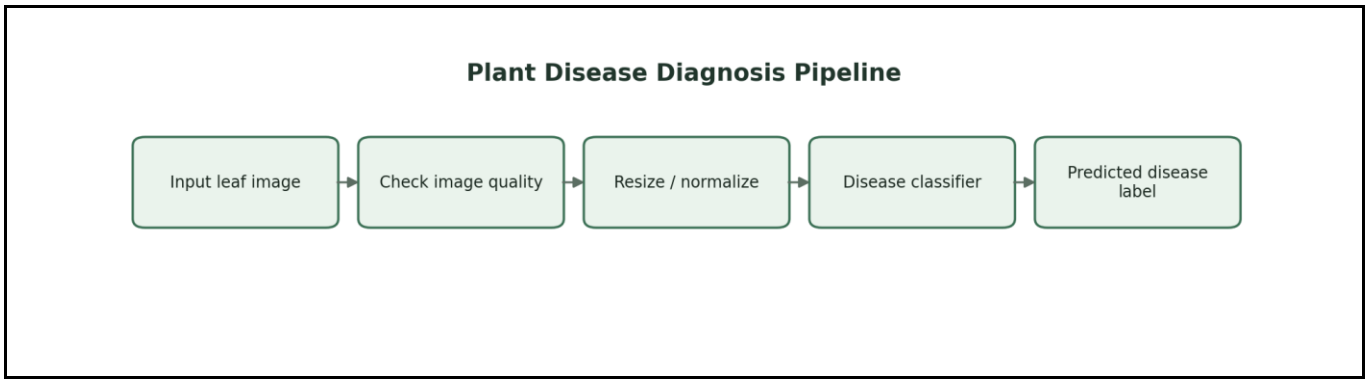
```
# Illustrative preprocessing structure
image = cv2.imread(image_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
image = cv2.resize(image, (INPUT_SIZE, INPUT_SIZE))
image = image.astype("float32") / 255.0
```

The exact preprocessing sequence must match the model used during training. The code above is an illustrative structure and is not presented as a claim that it is the exact training pipeline unless it matches the implemented project code.

4.1.2 PLANT IDENTIFICATION

Plant identification provides context for later diagnosis and recommendations. The identification module maps the visual input to one of the supported plant or crop classes. The output can then be used to constrain the disease search space and to present plant-specific care information.

If the image does not meet the supported class requirements, the interface should communicate that limitation instead of presenting an unsupported plant name as certain.



4.1.3 DISEASE DIAGNOSIS

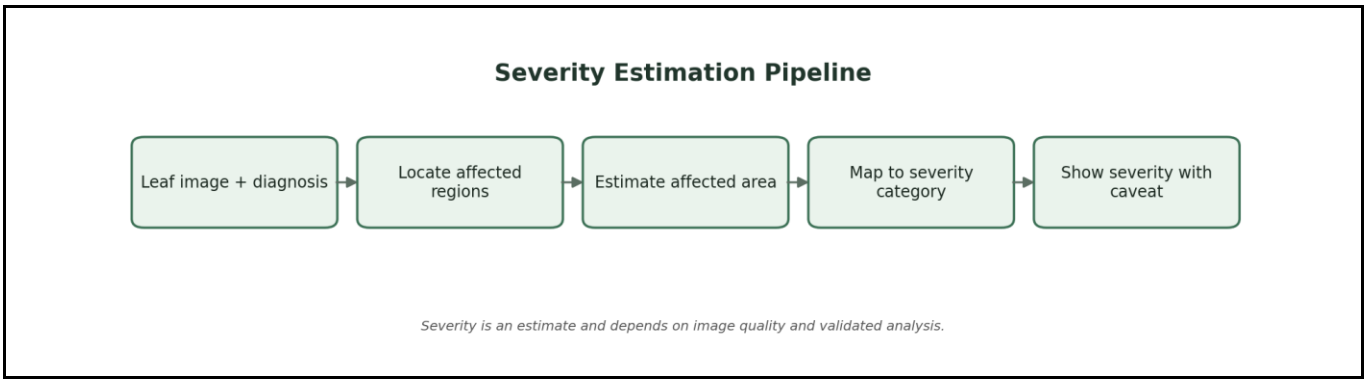
The disease diagnosis component receives the preprocessed image and applies the configured machine learning model. The model output is converted into a human-readable diagnosis. Where the model exposes confidence information, that information can be displayed as supporting evidence rather than as an absolute guarantee.

The diagnosis layer should also be designed to handle an unsupported or uncertain input. This is important because an image may contain multiple conditions, poor lighting, severe occlusion, or a disease not represented in the training classes.

4.1.4 LESION SEGMENTATION AND SEVERITY ANALYSIS

Severity estimation is intended to provide more information than a disease label. A segmentation component such as U-Net can attempt to identify lesion regions at pixel level. Classical HSV-based color segmentation can also be useful for selected visible symptoms where color differences provide a meaningful signal.

A simplified severity calculation can be expressed as the proportion of detected affected pixels relative to the analyzed plant or leaf region. The thresholds for labels such as Low, Moderate, and High must be determined from validated project data and should not be presented as universally correct without evaluation.



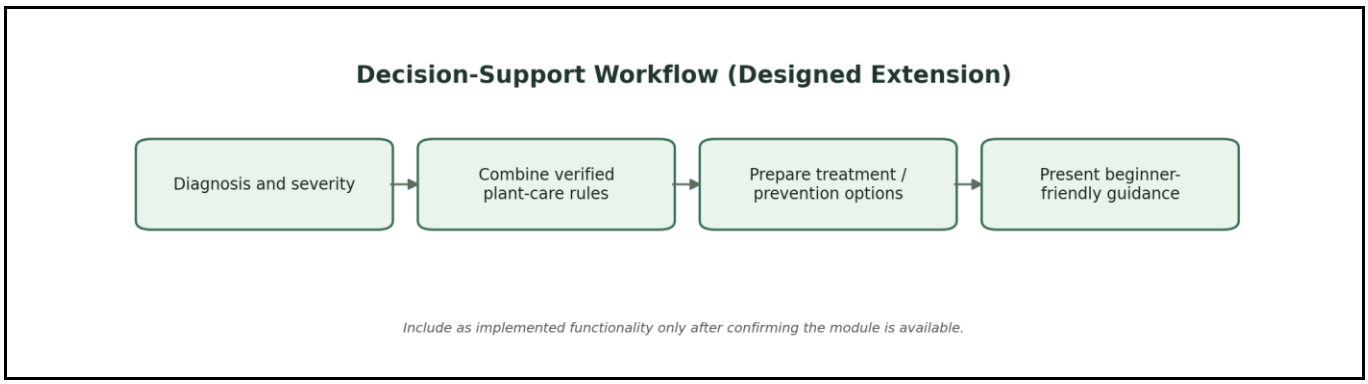
4.1.5 RISK ESTIMATION

Risk estimation combines available signals such as disease prediction, severity category, and user-provided context. The purpose is to help the user understand urgency. Risk should be communicated carefully because visual evidence alone may not capture environmental conditions, pathogen spread, or the presence of multiple simultaneous problems.

RISK LEVEL	GENERAL INTERPRETATION
Low	Limited visible concern; continue monitoring and preventive care.
Moderate	Action and closer observation are recommended.
High	Prompt intervention and expert confirmation may be appropriate.
Unknown	Evidence is insufficient for a reliable risk estimate.

4.1.6 RECOMMENDATION ENGINE

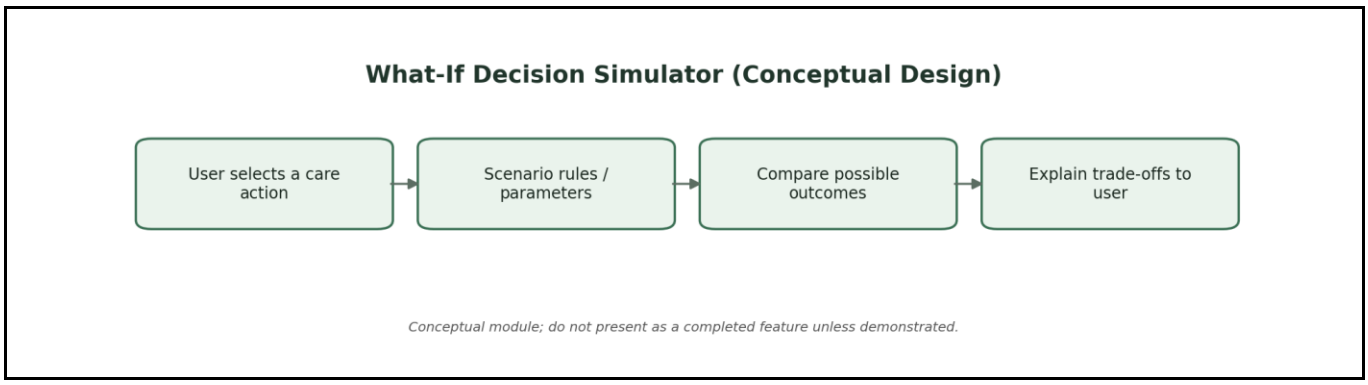
The recommendation engine converts the technical diagnosis into practical actions. Recommendations can be organized into immediate care, treatment options, prevention, monitoring, and escalation to expert advice. Where suitable, the interface can distinguish organic or low-impact options from conventional chemical options without implying that one option is universally correct.



4.1.7 AI WHAT-IF DECISION SIMULATOR

The What-If Decision Simulator is intended to make the system more interactive. Instead of only answering what the plant has, it allows a user to explore what may happen under alternative care decisions. Example scenario inputs can include increased monitoring, removal of affected leaves, environmental correction, preventive treatment, or delayed action.

The simulator should clearly distinguish between model-supported predictions and scenario explanations. It should not present speculative outcomes as guaranteed biological results.



4.1.8 RAG-BASED AGRICULTURAL ASSISTANT

The assistant module is designed for plant-related questions that may not be directly answered by an image classifier. A retrieval stage searches a controlled agricultural knowledge source for relevant information. The retrieved context is then supplied to the language model so that the response can remain grounded in the available material.

The separation of retrieval and generation is important because it allows the knowledge source to be reviewed and updated independently of the language model. The final interface should communicate when information is uncertain or when the user should consult a qualified agricultural professional.

4.2 IMPLEMENTATION

The backend is implemented using FastAPI. The main application connects multiple routers so that the project remains modular. The known project structure includes AI analysis, assistant, recommendation, and plant routers.

```
# AgriGuardian - Main FastAPI Application
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware


# Project routers
# from routers import ai_router, assistant_router
# from routers import recommendation_router, plant_router

app = FastAPI(
    title="Agri Guardian API",
    version="1.0.0",
    description="AI-powered plant diagnosis and decision support system"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Router registration is performed here in the implemented project.
```

The code listing above reflects the main FastAPI application structure supplied for the project. The internal implementations of the routers are not reproduced here because their complete source code was not provided in the source material used for this conversion.

4.2.1 AI ROUTER

The AI router is responsible for image-analysis requests. A typical request lifecycle includes receiving an uploaded image, validating the file, preprocessing the image, invoking the configured plant/disease model, applying severity analysis when available, and returning a structured response.

INPUT	PROCESSING	OUTPUT
Plant image	Validation and preprocessing	Processed image representation
Processed image	AI inference	Plant/disease prediction
Prediction + lesion data	Severity logic	Severity category / measurement where supported
Diagnosis + severity	Decision layer	Risk and recommendation context

4.2.2 RECOMMENDATION ROUTER

The recommendation router accepts diagnosis and contextual information and returns structured guidance. Separating recommendations from the AI model makes it possible to update care rules without retraining the visual classifier.

4.2.3 ASSISTANT ROUTER

The assistant router handles natural-language plant questions. It can connect the user query to the retrieval layer and return a grounded answer. The module can also preserve conversation context when supported by the application architecture.

4.2.4 PLANT ROUTER

The plant router provides plant-specific information and related operations. This can include supported plant classes, care information, identification metadata, and other structured plant resources.

4.2.5 FRONTEND INTEGRATION

The frontend communicates with the FastAPI backend through HTTP requests. The interface should provide a simple workflow: upload or capture an image, submit it for analysis, display the diagnosis, show severity/risk when available, present recommendations, and allow the user to ask follow-up questions.

4.3 TESTING AND VALIDATION

Testing was planned around module functionality, API communication, invalid input handling, and result presentation. Because the supplied source report did not contain AgriGuardian test measurements, the table below is a validation template that should be completed with the actual observed results before final submission.

TEST ID	INPUT / CONDITION	EXPECTED BEHAVIOUR	ACTUAL RESULT
TC01	Valid plant image	Image accepted and analysis initiated	[Enter actual result]
TC02	Unsupported file	System rejects or handles invalid input	[Enter actual result]
TC03	Supported disease image	Diagnosis returned	[Enter actual result]
TC04	Low-quality image	System warns or returns uncertain output	[Enter actual result]
TC05	Recommendation request	Relevant guidance returned	[Enter actual result]
TC06	Assistant question	Grounded response returned	[Enter actual result]
TC07	What-If scenario	Scenario response displayed	[Enter actual result]
TC08	API unavailable	Frontend shows controlled error	[Enter actual result]

4.3.1 MODEL VALIDATION

Model validation should be based on a held-out test set that is representative of the classes supported by the implementation. Appropriate metrics may include accuracy, precision, recall, F1-score, and a confusion matrix for classification. For segmentation, intersection-over-union or Dice-based measures can be used where ground-truth masks are available.

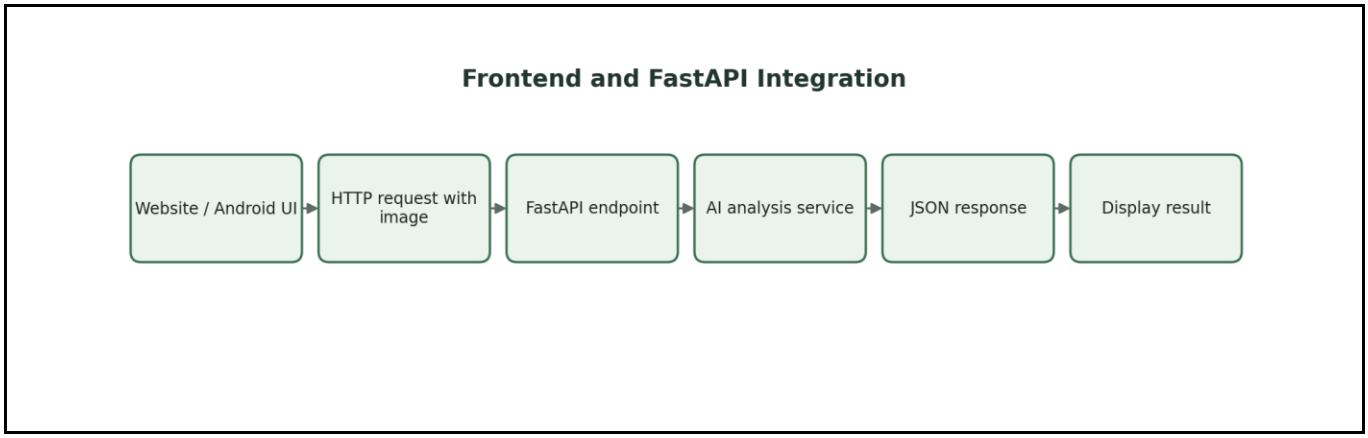
No AgriGuardian metric values are inserted in this converted report because the supplied hospital report does not provide valid AgriGuardian measurements. Actual project values should be inserted from the trained model evaluation output.

METRIC	AGRI GUARDIAN VALUE
Accuracy	[INSERT ACTUAL VALUE]
Precision	[INSERT ACTUAL VALUE]
Recall	[INSERT ACTUAL VALUE]
F1-score	[INSERT ACTUAL VALUE]
Segmentation IoU / Dice	[INSERT IF ACTUALLY IMPLEMENTED]

4.4 INTEGRATION

Integration testing checks whether the individual modules operate correctly when connected. The main integration path is Frontend → FastAPI → AI preprocessing → diagnosis → severity/risk → recommendations → Frontend. Assistant queries follow Frontend → Assistant Router → Retrieval → Generation → Frontend.

The modular design also reduces the effect of changes. For example, replacing the disease model does not require rewriting the assistant or recommendation router as long as the API response contract remains stable.


Frontend and FastAPI Integration
Website / Android UI
HTTP request with image
FastAPI endpoint
AI analysis service
JSON response
Display result

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

AgriGuardian presents an AI-powered approach to plant health diagnosis and decision support for users who may not have formal agricultural knowledge. The project combines computer vision and machine learning with a modular backend architecture to provide a more complete workflow than a simple disease classifier.

The system is designed to begin with an image and progress toward an actionable response through plant identification, disease diagnosis, severity analysis, risk interpretation, recommendations, prevention guidance, and interactive assistance. The RAG assistant and What-If Decision Simulator further extend the project from prediction toward decision support.

The FastAPI architecture provides a practical service layer for the demonstration website and creates a foundation for future mobile application integration. The final performance of the system
depends on the actual trained models, datasets, validation results, and knowledge sources used
during implementation.

5.2 LIMITATIONS

- Image quality, lighting, background clutter and occlusion can affect visual diagnosis.
- Model performance is limited by the coverage and quality of the training dataset.
- Multiple simultaneous diseases may be difficult to classify from a single image.
- Severity estimation requires reliable lesion or affected-region information.
- Risk estimation from an image alone cannot capture every environmental factor.
- Recommendations should be treated as decision support and not as guaranteed treatment instructions.
- The RAG assistant is dependent on the quality, coverage and maintenance of its knowledge source.
- Actual performance metrics must be obtained from the implemented AgriGuardian models rather than transferred from another project.

5.3 FUTURE WORK

- Expand the supported crop and disease classes using larger, diverse datasets.
- Add robust multi-disease and multi-lesion detection.
- Improve segmentation with validated U-Net or other segmentation architectures.
- Integrate environmental sensors such as temperature, humidity and soil moisture.

- Add temporal plant-health tracking across repeated images.
- Develop a native Android application using the existing FastAPI services.
- Improve uncertainty estimation and out-of-distribution detection.
- Add expert-review or escalation workflows.
- Improve multilingual support for beginner users.
- Deploy optimized models for edge or mobile inference where feasible.
- Add explainable AI visualizations showing which regions influenced a prediction.
- Expand the What-If simulator using validated agronomic rules and historical observations.

5.4 SOURCE CODE

The original supplied report contained hospital-specific training and web-interface source code. Those listings are not reproduced as AgriGuardian code because doing so would falsely represent the hospital implementation as the present project. The following structure should be used for the final AgriGuardian source-code appendix, populated from the actual project repository.

FILE / MODULE	PURPOSE	STATUS
main.py	FastAPI application and router registration	Partially documented from supplied code
ai_router	Image analysis API endpoints	Insert actual project source
assistant_router	RAG assistant endpoints	Insert actual project source
recommendation_router	Recommendation endpoints	Insert actual project source
plant_router	Plant information/identification endpoints	Insert actual project source
preprocessing module	OpenCV image preprocessing	Insert actual project source
model module	Plant/disease model loading and inference	Insert actual project source
segmentation module	U-Net / HSV lesion processing if implemented	Insert actual project source
frontend files	User interface and API integration	Insert actual project source

```
# Recommended final source-code appendix structure


agri_guardian/
├── main.py
├── routers/
│   ├── ai_router.py
│   ├── assistant_router.py
│   ├── recommendation_router.py
│   └── plant_router.py
├── services/
│   ├── preprocessing.py
│   ├── diagnosis.py
│   ├── severity.py
```

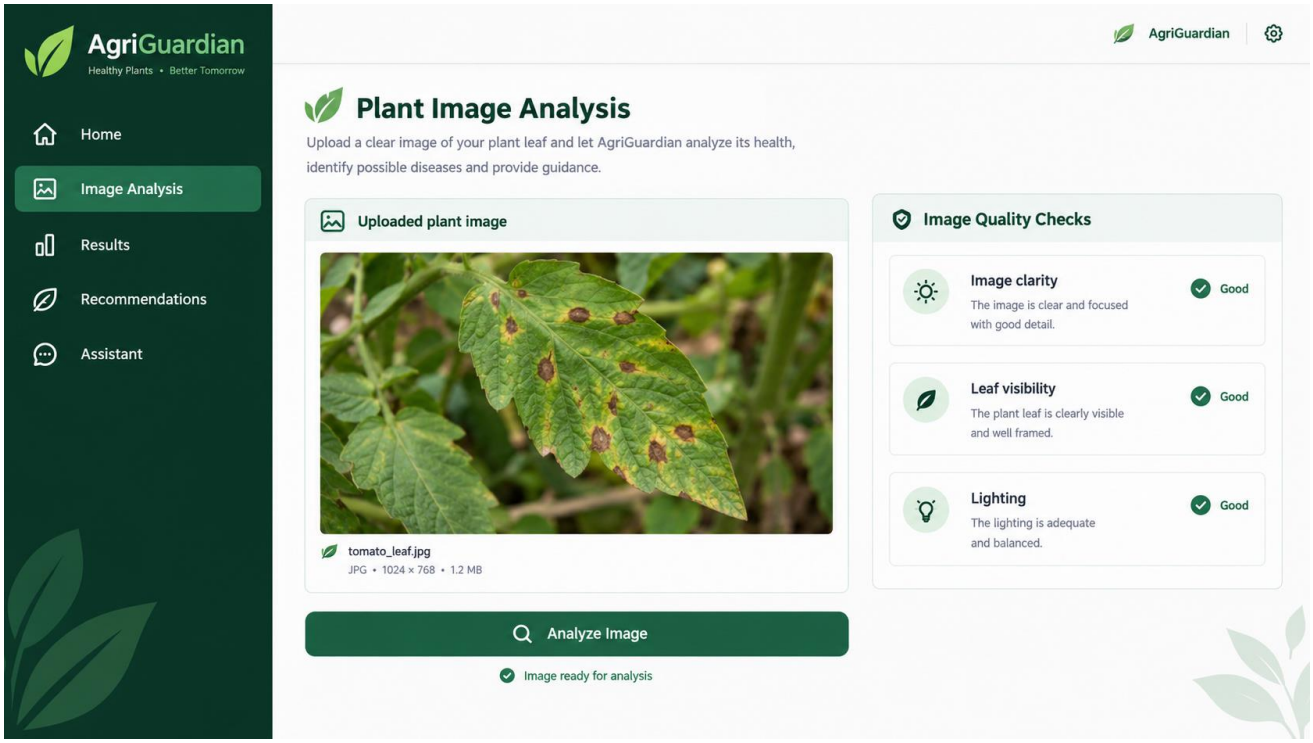
```
│   ├── risk.py
│   ├── recommendation.py
│   └── rag_assistant.py
├── models/
│   └── [actual trained model files]
├── frontend/
│   └── [actual frontend files]
└── requirements.txt
```

The exact contents of these files should be copied from the working AgriGuardian implementation before submission. This appendix intentionally avoids inventing code that was not supplied in the source material.

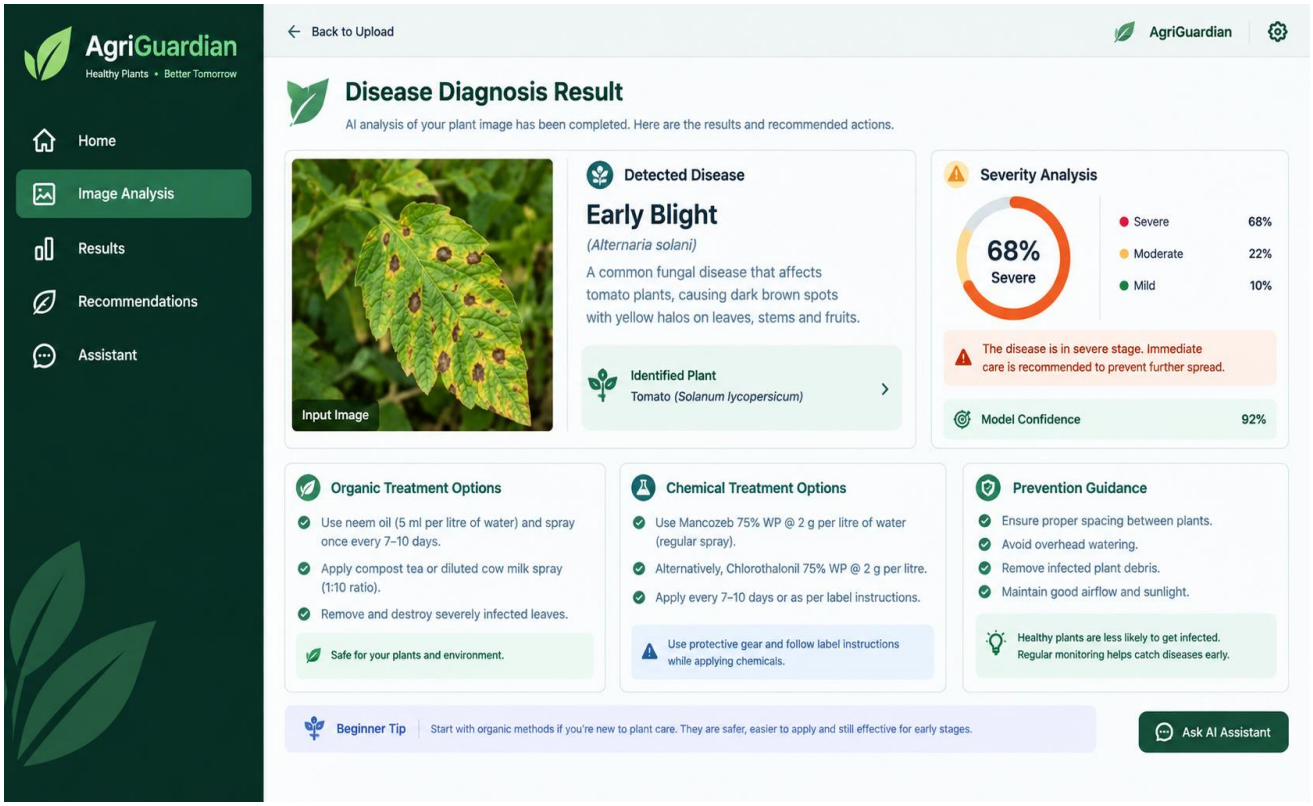
5.4 RESULTS

The results section should document the actual demonstration of AgriGuardian. Insert screenshots captured from the working application in the placeholders below. Do not use screenshots from unrelated projects.

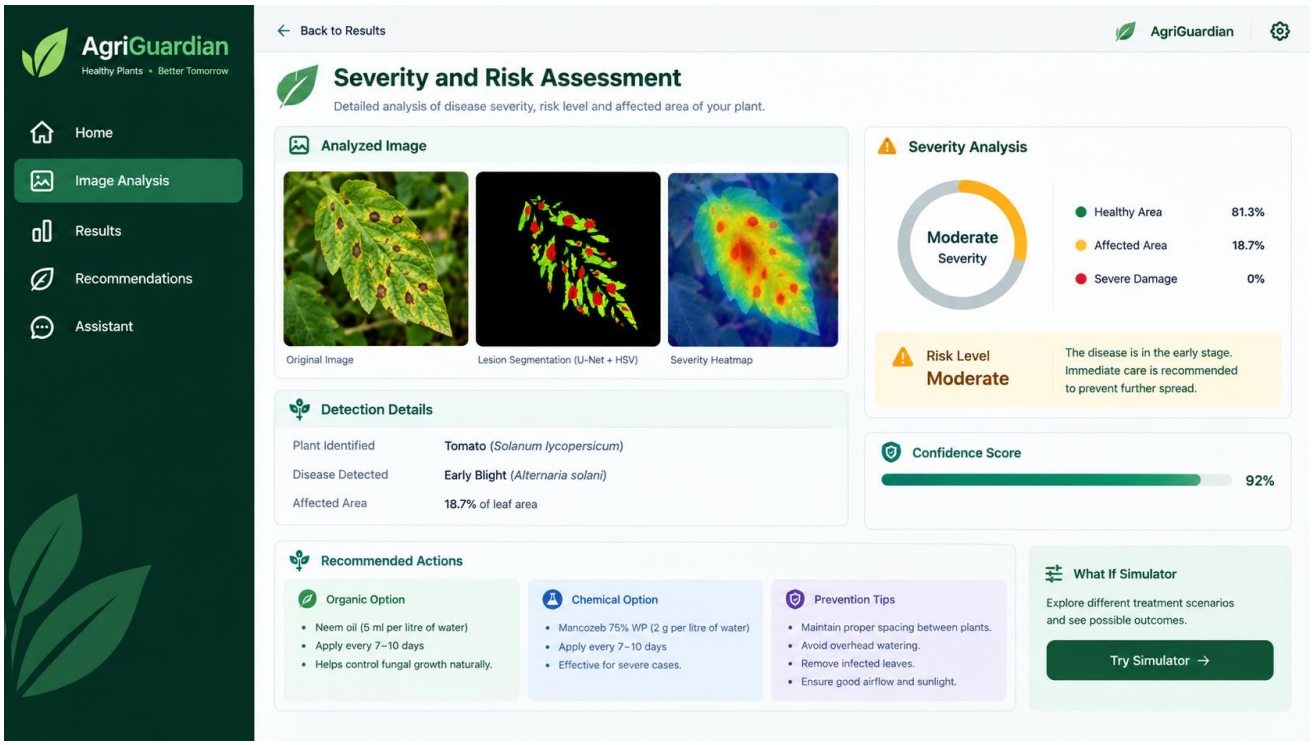
[AGRIGUARDIAN USER INTERFACE / IMAGE ANALYSIS SCREENSHOT]



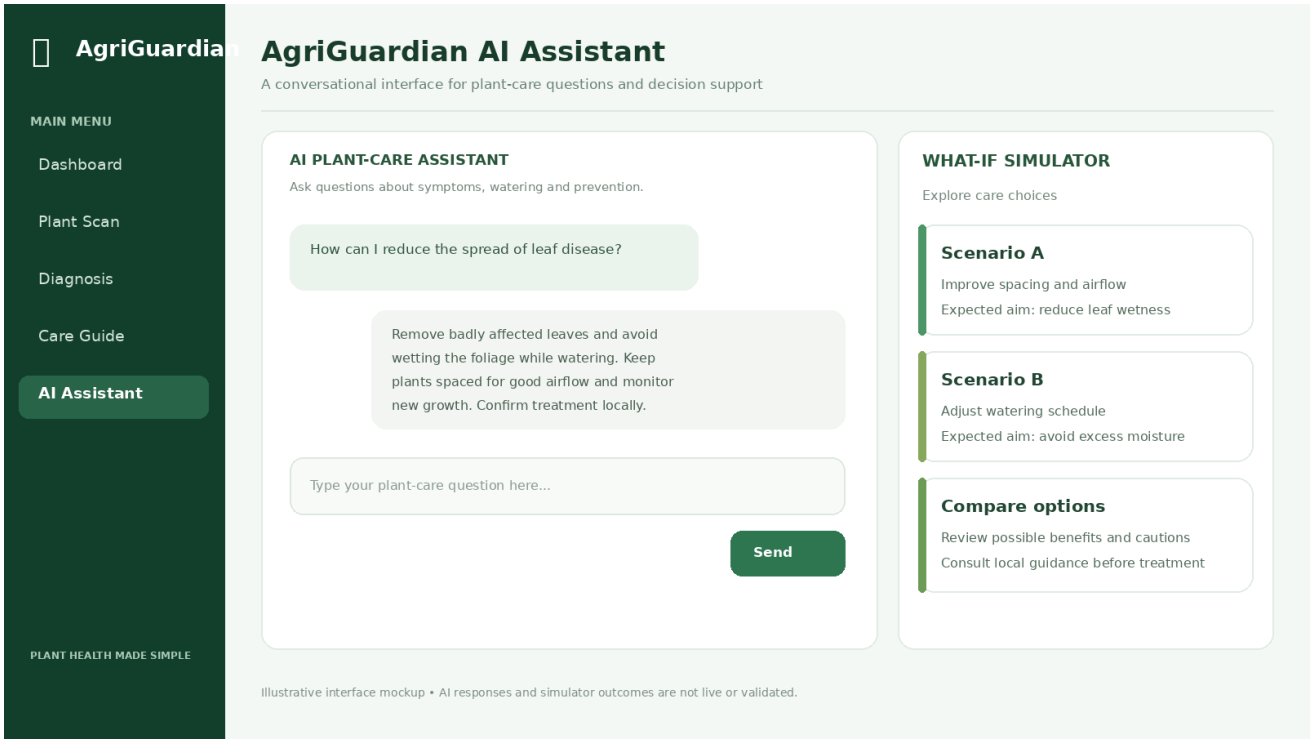
[FIGURE 9 – DISEASE DIAGNOSIS RESULT SCREENSHOT]



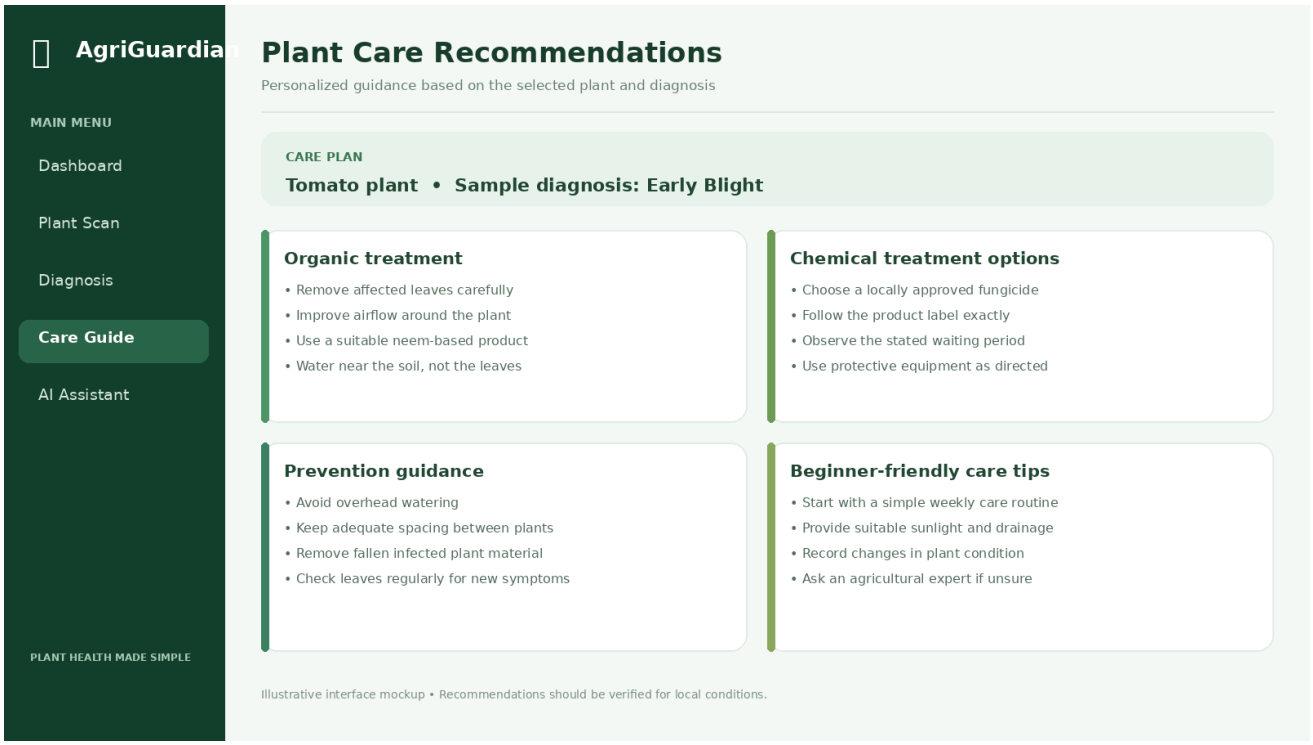
[FIGURE 10 – SEVERITY AND RISK ASSESSMENT SCREENSHOT]



[FIGURE 11 – RECOMMENDATION SCREENSHOT]



[FIGURE 12 – RAG ASSISTANT / WHAT-IF SIMULATOR SCREENSHOT]



5.4.1 RESULT INTERPRETATION

The final result interpretation should describe what the implemented system actually produced during demonstration. For each screenshot, record the input condition,

predicted plant/disease class, confidence if exposed, severity if implemented, risk category if implemented, and recommendation output.

ITEM	VALUE TO ENTER FROM ACTUAL DEMO
Plant / crop	Tomato
Disease / condition	Early Blight
Confidence	92
Severity	Moderate
Risk	Moderate
Recommendation	The disease is in the early stage. Immediate care is recommended to prevent further spread.
Assistant response	Remove badly affected leaves and avoid wetting the foliage while watering

REFERENCES:

1. PlantVillage dataset and plant disease image classification literature – use the exact dataset/publication actually used by the project.

2. Mohanty, S. P., Hughes, D. P., and Salathé, M., "Using Deep Learning for Image-Based Plant Disease Detection," Frontiers in Plant Science, 2016.

3. Ronneberger, O., Fischer, P., and Brox, T., "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015.

4. He, K., Zhang, X., Ren, S., and Sun, J., "Deep Residual Learning for Image Recognition," CVPR, 2016.

5. Tan, M. and Le, Q. V., "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.

6. OpenCV documentation and project documentation for image processing operations used in implementation.

7. FastAPI documentation for API routing, request handling, middleware, and application integration.

8. Use the exact agricultural knowledge sources and RAG documents included in the implemented AgriGuardian knowledge base.

9. Add the actual model/dataset papers used by the team before final submission.

Important: the final reference list should be reconciled with the exact datasets, models, papers, documentation, and knowledge sources actually used by the implemented project. References from the original hospital report have not been silently presented as AgriGuardian evidence.